

Jakub

Osoba 4 - Rdzen silnika, integracja, logika gry, HUD

Materiały do obrony projektu: Silnik 3D + Strzelnica (FreeGLUT / OpenGL)

Twoje pliki: Engine/Engine.*, main.cpp, CMakeLists.txt, Demo/Gameplay.cpp, Demo/ShootingGallery.h, Demo/Constants.h

0. Gdzie sa moje pliki (mapa do prezentacji)

Strzałka <<< pokazuje pliki, które robiłem ja (Jakub).

```
pgk/
|
+-- main.cpp          <<< MOJE  (start programu, spina wszystko)
+-- CMakeLists.txt    <<< MOJE  (konfiguracja kompilacji)
+-- README.md Doxyfile .gitignore <<< MOJE  (infrastruktura)
|
+-- Engine/
|   +-- Math3D.* SceneNode.*      (Lukasz)
|   +-- PrimitiveNode.*          (Rogert)
|   +-- Camera.* Light.* Texture.* (Kacper)
|   +-- Engine.h / Engine.cpp    <<< MOJE  (okno, petla, wejscie, callbacki)
|
+-- Demo/
|   +-- Room.cpp Obstacles.cpp Targets.cpp  (Lukasz/Rogert/Kacper)
|   +-- ShootingGallery.h <<< MOJE  (deklaracja calej klasy gry)
|   +-- Constants.h      <<< MOJE  (stale gry - balans)
|   +-- Gameplay.cpp     <<< MOJE  (logika gry, strzelanie, HUD)
```

W skrocie: ja jestem 'spinaczem'. Robie **rdzen silnika** (okno przez FreeGLUT, petla rysowania, obsługa klawiatury i myszy) oraz **glowna logike gry** (co sie dzieje co klatke, strzelanie, punkty, zycia, HUD). Naglowek ShootingGallery.h jest 'kontraktem' - reszta zespolu implementuje swoje funkcje, które ja tu zadeklarowalem.

1. O co chodzi w moich plikach (po ludzku)

Engine to silnik bez gry - sam w sobie tylko otwiera okno, ustawia OpenGL i kreci petla: 'policz -> narysuj -> powtorz'. Gra podpinia sie do niego przez **callbacki** (funkcje zwrotne): silnik mowi 'mamy nowa klatke' i wola moja funkcje gry.

Gameplay.cpp to mozg gry: porusza graczem, sprawdza strzaly, liczy punkty i zycia, rysuje HUD (celownik, wyniki, ekran konca gry). **Constants.h** trzyma wszystkie liczby balansu w jednym miejscu (np. ile zyc, jak dlugo zyje cel).

2. Engine - serce silnika

2.1. Petla czasu (zmienne FPS)

FreeGLUT wola moja funkcje onTimer co pewien czas. Licze **dt** (czas od ostatniej klatki) i przekazuje go do gry - dzieki temu ruch jest plynny niezaleznie od FPS.

Engine.cpp - petla czasu z dt i pauza

```
void Engine::onTimer() {
    int now = glutGet(GLUT_ELAPSED_TIME);
    if (paused) { lastTickMs = now; glutPostRedisplay();
                glutTimerFunc(1000/targetFPS, timerCallback, 0); return; }
    float dt = (now - lastTickMs) / 1000.0f; // sekundy od ostatniej klatki
    lastTickMs = now;
    ... // ruch kamery silnika (gdy tryb ENGINE_ORBIT)
    if (updateCallback) updateCallback(dt); // <- logika gry
    glutPostRedisplay(); // popros o przerysowanie
    glutTimerFunc(1000/targetFPS, timerCallback, 0); // zaplanuj kolejna klatke
}
```

Klawisze +/- zmieniaja targetFPS, czyli jak czesto planujemy klatke - to spelnia wymaganie 'zmienna szybkoosc odswiezania'. Pauza (ESC) pomija logike gry, ale dalej rysuje (zeby pokazac ekran PAUZA).

2.2. Obsluga wejscia + free mouse look

Klawiatura: trzymam tablice keys[256] (true gdy wcisniety). Gra pyta isKeyDown('w'). Mysz w trybie FPP chowam kursor i po kazdym ruchu przesuwam go z powrotem na srodek ekranu - dzieki temu mozna sie obracac w nieskonczonosc.

Engine.cpp - rozgladanie sie myszka (uproszczone)

```
void Engine::handleMouseMotion(int x, int y) {
    if (paused) return;
    if (freeMouseLook) {
        int cx = windowWidth/2, cy = windowHeight/2;
        if (x==cx && y==cy) return; // ignoruj wlasny warp
        cameraYaw += (x-cx) * mouseSensitivity;
        cameraPitch += -(y-cy) * mouseSensitivity;
        cameraPitch = clamp(cameraPitch, -85deg, +85deg);
        glutWarpPointer(cx, cy); // wroc kursor na srodek
    }
}
```

2.3. Callbacki - jak silnik laczy sie z gra

Silnik nie wie nic o 'strzelnicy'. Trzyma tylko wskazniki na funkcje (std::function), ktore gra mu daje. To rozdziela silnik od konkretnej gry.

main.cpp - podpiecie logiki gry pod silnik

```
// w main.cpp - gra rejestruje sie w silniku:  
engine.setUpdateCallback([&](float dt){ game.onUpdate(dt); });  
engine.setShootCallback ([&](){ game.onShoot(); });  
engine.setHUDCallback    ([&](){ game.onHUD(); });  
engine.setResetCallback ([&](){ game.onReset(); });
```

2.4. Dlaczego callbacki GLUT sa statyczne

FreeGLUT to biblioteka w C - przyjmuje zwykle funkcje (wskazniki), nie metody klasy. Dlatego mam statyczne funkcje, ktore przez wskaznik 'instance' wolaja prawdziwe metody obiektu Engine.

Engine.cpp - mostek miedzy C-owym GLUT a obiektem C++

```
static Engine* instance; // jedyny obiekt silnika  
  
void Engine::displayCallback() { if (instance) instance->render(); }  
void Engine::timerCallback(int){ if (instance) instance->onTimer(); }  
// GLUT dostaje displayCallback, a ta woła instance->render()
```

3. Demo/Gameplay.cpp - logika gry

Tu jest konstruktor gry (ustawia swiatla, wola buildRoom/buildObstacles itd. od kolegow, konfiguruje kamere FPP) oraz 4 glowne funkcje: onUpdate, onShoot, onHUD, onReset.

3.1. onShoot - strzelanie (raycast)

Gameplay.cpp - rdzen strzelania

```
void ShootingGallery::onShoot() {
    if (gameOver_ || targets_.empty()) return;
    // odswiez kamere NAJNOWSZYM yaw/pitch (mysz mogla sie ruszyc!)
    engine_.getCamera()->setFirstPerson(playerEye_,
        engine_.getCameraYaw(), engine_.getCameraPitch());
    Vec3 origin = engine_.getCamera()->eyePosition();
    Vec3 dir    = engine_.getCamera()->lookDirection();

    float closestT = MAX_SHOOT_DIST; int hitIdx = -1;
    for (size_t i=0; i<targets_.size(); ++i) {
        float t;
        if (raySphereIntersect(origin, dir, currentTargetPos(targets_[i]),
            TARGET_RADIUS*HIT_TOLERANCE, t)
            && t > 0 && t < closestT) { closestT = t; hitIdx = i; }
    }
    if (hitIdx >= 0 && obstacleBlocksRay(origin, dir, closestT)) hitIdx = -1;
    // hitIdx >= 0 -> trafienie; inaczej pudlo (-1 zycie)
}
```

Najwazniejszy szczegol: przed raycastem odswiezam kamere najnowszym yaw/pitch. To naprawilo bug, gdzie strzaly mijaly cel - mysz mogla sie ruszyc miedzy klatka a wcisnieciem spacji, wiec kierunek byl nieaktualny. **HIT_TOLERANCE** (1.4) lekko powieksza cel, zeby latwiej bylo trafic.

3.2. onUpdate - co klatke

Porusza graczem (WASD), wola kolizje (Rogert), aktualizuje kamere, odlicza timery efektow, sprawdza czy cele nie wygasly, respawnuje fale. Wszystko skalowane przez **dt**, zeby tempo bylo niezalezne od FPS.

Gameplay.cpp - ruch gracza z dt

```
float speed = PLAYER_MOVE_SPEED * dt; // metry = predkosc * czas
if (engine_.isKeyDown('w')) playerEye_ += fwd * speed;
...
applyObstacleCollision(); // wypchnij z przeszkod
playerEye_.x = clamp(playerEye_.x, sciany...); // nie wychodz z pokoju
```

3.3. onHUD - interfejs 2D

Rysuje na plasko (2D): celownik na srodku, wyniki/zycia/czas w rogu, pasek czasu celu, czerwony/zielony blysk po pudle/trafieniu, ekran Game Over i Pauza. Uzywam pomocnikow drawString z silnika.

4. Constants.h i ShootingGallery.h

Constants.h to wszystkie liczby balansu w jednym miejscu (namespace sg): rozmiar pokoju, ile zyc, jak dlugo zyje cel, predkosc gracza. Zmiana trudnosci = zmiana jednej liczby. Uzywam constexpr, ktore w C++17 sa inline - mozna wlaczac naglowek w wielu plikach bez bledu linkera.

ShootingGallery.h deklaruje cala klase gry: struktury (Target, Obstacle...), wszystkie pola i metody. To 'kontrakt' - ja go pisze, a koledzy implementuja swoje metody (buildRoom, buildObstacles, spawnWave) w swoich plikach .cpp. Dzieki temu mozemy pracowac rownolegle bez kolizji w gicie.

5. CMakeLists.txt - jak to sie kompiluje

CMake zbiera wszystkie pliki .cpp, znajduje OpenGL i FreeGLUT (rozne sciezki dla Windows/macOS/Linux) i tworzy plik wykonywalny. Dodatkowo kopiuje folder assets (tekstury BMP) i freeglut.dll obok exe, zeby gra je znalazla.

CMakeLists.txt - szkielet

```
set(ENGINE_SOURCES Engine/Engine.cpp Engine/Math3D.cpp ...)  
set(DEMO_SOURCES   Demo/Room.cpp Demo/Obstacles.cpp  
                  Demo/Targets.cpp Demo/Gameplay.cpp)  
add_executable(${PROJECT_NAME} main.cpp ${ENGINE_SOURCES} ${DEMO_SOURCES})  
# osobny target kopiuje assets/ przy KAZDYM buildzie (ALL)  
add_custom_target(copy_assets ALL COMMAND ... copy_directory assets ...)
```

6. Pytania od wykładowcy - przygotowanie

Przy każdym pytaniu jest pasek **Gdzie** - mówi w którym pliku i w której funkcji szukać odpowiedzi, żebyś w trakcie obrony szybko otworzył właściwe miejsce.

Engine - inicjalizacja

P: Co robi setGraphicsParams i dlaczego dopiero po nim można używać OpenGL?

O: Tworzy okno (glutCreateWindow) i kontekst OpenGL, ustawia tryb wyświetlania (depth, double buffer), włącza oświetlenie i rejestruje callbacki. Dopiero gdy istnieje kontekst GL, można wgrywać tekstury - dlatego gra tworzy tekstury PO tym wywołaniu.

Gdzie: Engine.cpp -> setGraphicsParams(); main.cpp (kolejność wywołania)

P: Co to jest double buffering i depth buffer?

O: Double buffering: rysujemy do ukrytego bufora, a na koniec klatki podmieniamy go z widocznym (glutSwapBuffers) - dzięki temu nie widać rysowania w trakcie, obraz nie miga. Depth buffer (z-buffer): pamięta głębokość każdego piksela, żeby bliższe obiekty zasłaniały dalsze.

Gdzie: Engine.cpp -> setGraphicsParams() (GLUT_DOUBLE, GLUT_DEPTH); render() (glutSwapBuffers)

Engine - petla i czas

P: Jak działa petla główna w waszym silniku?

O: FreeGLUT po glutMainLoop wola zarejestrowane callbacki. timerCallback (onTimer) odpala się co $1000/\text{targetFPS}$ ms - tam liczy Δt , aktualizuje logikę i woła glutPostRedisplay, które prosi o przerysowanie (displayCallback -> render). Potem planuje kolejny timer. Tak kręci się petla 'licz -> rysuj'.

Gdzie: Engine.cpp -> onTimer(), render(), run() (glutMainLoop)

P: Co to jest Δt i dlaczego mnożysz przez nie ruch?

O: Δt to czas od poprzedniej klatki w sekundach: $(\text{now} - \text{lastTickMs})/1000$. Mnożąc prędkość przez Δt , dostajemy faktyczny dystans (droga = prędkość * czas). Dzięki temu przy 30 i 120 FPS gracz porusza się tak samo szybko - inaczej przy wyższym FPS ruszałby się szybciej.

Gdzie: Engine.cpp -> onTimer() (float dt = (now-lastTickMs)/1000.0f)

P: Jak realizujecie zmienną szybkość odświeżania?

O: targetFPS steruje interwałem timera (glutTimerFunc(1000/targetFPS,...)). Klawisze +/- zmieniają targetFPS o 10 (z ograniczeniem 5..240). Zmierzone i docelowe FPS pokazuje na HUD. To pokazuje, że petla jest sterowalna - spełnia wymaganie 'zmienna szybkość odświeżania'.

Gdzie: Engine.cpp -> handleKeyboard() (case '+'/'-'); drawHUD() (FPS); onTimer() (glutTimerFunc)

P: Cemu klawisze +/- obsugujesz w handleKeyboard, a nie sprawdzasz keys[] co klatke?

O: Bo sprawdzanie keys['+'] co klatke przy 60 FPS zmienialoby FPS 60 razy na sekunde - jedno naciśnięcie skakaloby o duzo. handleKeyboard odpala sie RAZ na naciśnięcie klawisza, wiec kazde wciśnięcie to dokładnie jedna zmiana o 10. To byl realny bug, który naprawilem.

Gdzie: Engine.cpp -> handleKeyboard() (case '+': case '=': setTargetFPS(...))

P: Jak dziala pauza?

O: Flaga paused. W onTimer, gdy paused, pomijam logike gry (nie wolam updateCallback), ale dalej rysuje (glutPostRedisplay) zeby pokazac napis PAUZA. Aktualizuje lastTickMs, zeby po wznowieniu dt nie bylo ogromne (inaczej gracz 'przeskoczyly'). ESC przelacza pauze, Q w pauzie wychodzi.

Gdzie: Engine.cpp -> onTimer() (if paused), togglePause(); handleKeyboard() (case 27, case 'q')

Engine - wejscie i mysz

P: Jak obslugujesz klawiature - czemu tablica keys[256]?

O: GLUT wola keyboardDown gdy klawisz wciśnięty i keyboardUp gdy puszczoney. Zapisuje keys[key] = true/false. Dzieki temu gra moze pytac isKeyDown('w') w dowolnej chwili (ruch ciagly). Akcje jednorazowe (strzal, pauza, FPS) obsuguje od razu w handleKeyboard.

Gdzie: Engine.cpp -> handleKeyboard() (keys[key]=isDown); isKeyDown() w Engine.h

P: Jak dziala free mouse look?

O: Chowam kursor (glutSetCursor NONE). Po kazdym ruchu liczę przesuniecie wzgledem srodka ekranu (x-cx, y-cy), dodaje do yaw/pitch, ograniczam pitch, a potem przesuwam kursor z powrotem na srodek (glutWarpPointer). Dzieki temu kursor nigdy nie dochodzi do krawedzi i mozna obracac sie w nieskonczonosc.

Gdzie: Engine.cpp -> handleMouseMotion() (galaz freeMouseLook), setFreeMouseLook()

P: Po co warunek if (x==cx && y==cy) return w handleMouseMotion?

O: Bo glutWarpPointer (przesuniecie kursora na srodek) SAM generuje zdarzenie ruchu myszy. Bez tego warunku zdarzenie z warpu bylyby liczone jako ruch gracza i kamera 'drgalaby'. Ignoruje wiec zdarzenie, gdy kursor jest dokładnie na srodku (czyli to nasz wlasny warp).

Gdzie: Engine.cpp -> handleMouseMotion() (if x==cx && y==cy return)

P: Cemu LPM strzela tylko w trybie freeMouseLook?

O: W trybie FPP (gra) lewy przycisk = strzal. W trybie orbity (test silnika) lewy przycisk sluzi do obracania kamery przeciaganiem. Wiec strzelam tylko, gdy freeMouseLook jest wlaczony - inaczej LPM ma inne zadanie.

Gdzie: Engine.cpp -> handleMouse() (if freeMouseLook && shootCallback)

Engine - callbacki i architektura

P: Dlaczego callbacki GLUT sa statyczne?

O: FreeGLUT jest biblioteka C - przyjmuje wskazniki na zwykłe funkcje, nie metody klasy (metoda ma ukryty parametr this, wiec nie pasuje do typu zwykłej funkcji). Robie wiec statyczne funkcje, ktore przez globalny wskaznik 'instance' wolaja prawdziwe metody obiektu Engine.

Gdzie: Engine.cpp -> displayCallback(), timerCallback() itd.; static Engine* instance

P: Co to jest wskaznik instance i kiedy jest ustawiany?

O: To statyczny wskaznik na jedyny obiekt Engine, ustawiany w konstruktorze (instance = this). Statyczne callbacki uzywaja go, zeby dotrzec do metod instancji. To podejscie singleton-like - zakladamy jeden silnik na program.

Gdzie: Engine.cpp -> konstruktor Engine() (instance = this)

P: Po co callbacki (std::function) zamiast wpisac logike gry wprost w silnik?

O: Zeby silnik byl niezalezny od konkretnej gry. Engine wie tylko 'jest nowa klatka' i wola podana funkcje. Mozna podpiac dowolna gre bez zmiany silnika - luzne sprzezenie. Gra rejestruje swoje metody przez setUpdateCallback itd. w main.cpp.

Gdzie: Engine.h -> setUpdateCallback itd.; main.cpp -> rejestracja lambd

P: Co to jest CameraControlMode?

O: Przełącznik kto rzadzi kamera. ENGINE_ORBIT - silnik sam obraca kamera z WASD (tryb testowy). GAME_CONTROLLED - silnik nie rusza kamery, robi to gra w onUpdate (FPP gracza). Gra ustawia GAME_CONTROLLED, zeby silnik nie nadpisywal jej kamery.

Gdzie: Engine.h -> enum CameraControlMode; Engine.cpp -> onTimer() (if ENGINE_ORBIT)

P: Jak rysujesz HUD (2D) na scenie 3D?

O: Po narysowaniu sceny 3D przełączam projekcje na ortogonalna (Mat4::orthographic od Lukasa) i wyłączam depth test oraz oświetlenie. Wtedy rysuje plaskie elementy (tekst, paski) we wspolrzednych pikseli. Na koniec przywracam stan. Gra dorysowuje swoj HUD przez hudCallback.

Gdzie: Engine.cpp -> drawHUD(); drawString()/drawStringLarge()

Gameplay - logika i strzelanie

P: Wytłumacz krok po kroku co dzieje sie przy strzale.

O: 1) Odswiezam kamere najnowszym yaw/pitch. 2) Biore origin (oko) i dir (kierunek). 3) Dla kazdego celu robie raySphereIntersect i szukam najblizszego trafionego (najmniejsze t). 4) Sprawdzam, czy zadna przeszkoda nie jest blizej (obstacleBlocksRay) - jak tak, to pudlo. 5) Trafienie: +1 punkt, +seria, znika cel, bonus +2s dla reszty fali. Pudlo: -1 zycie.

Gdzie: Gameplay.cpp -> onShoot()

P: Dlaczego odswiezasz kamere przed raycastem - to byl wasz bug?

O: Tak. Kamera (yaw/pitch) byla aktualizowana raz na klatke w onUpdate. Ale gracz mogl ruszyc mysz i wcisnac spacje MIEDZY klatkami - wtedy raycast szedl w stary kierunek, mimo ze celownik wskazywal nowy. Strzaly mijaly cel. Naprawa: tuz przed raycastem ustawiam kamere aktualnymi katami getCameraYaw/Pitch.

Gdzie: Gameplay.cpp -> onShoot() (setFirstPerson na poczatku)

P: Jak znajdujesz, ktory cel zostal trafiony przy wielu celach?

O: Przechodze petla po wszystkich celach, robie raySphereIntersect dla kazdego i zapamietuje ten z najmniejszym t (najblizszy do kamery, czyli ten 'na pierwszym planie'). Dopiero potem sprawdzam czy przeszkoda go nie zaslania.

Gdzie: Gameplay.cpp -> onShoot() (petla szukajaca closestT, hitIdx)

P: Co to jest HIT_TOLERANCE?

O: Mnoznik (1.4), ktorem powiekszam promien celu TYLKO na potrzeby trafienia (raySphereIntersect dostaje TARGET_RADIUS*HIT_TOLERANCE). Dzieki temu mozna celowac troche obok srodka i nadal trafic - gra mniej frustruje. Wizualnie cel zostaje tej samej wielosci.

Gdzie: Gameplay.cpp -> onShoot() (TARGET_RADIUS*HIT_TOLERANCE); Constants.h -> HIT_TOLERANCE

P: Jak dziala bonus +2s po trafieniu?

O: Gdy w fali zostaja jeszcze cele, do kazdego dodaje 2 sekundy zycia - przez przesuniecie ich spawnTime do PRZODU (spawnTime += WAVE_HIT_BONUS). Timeout liczy sie jako totalTime - spawnTime, wiec przesuniecie spawnTime odracza wygasniecie. Prosty trik bez dodatkowego pola.

Gdzie: Gameplay.cpp -> onShoot() (petla remaining.spawnTime += WAVE_HIT_BONUS)

P: Jak liczysz ruch gracza i czemu skalujesz przez dt?

O: Z kata yaw licze wektory przod (fwd) i prawo (rgt) w poziomie. WASD dodaja je do pozycji oka skalowane przez speed = PLAYER_MOVE_SPEED * dt. Mnozenie przez dt sprawia, ze predkosc jest stala niezaleznie od FPS (droga = predkosc * czas).

Gdzie: Gameplay.cpp -> onUpdate() (fwd, rgt, speed = PLAYER_MOVE_SPEED*dt)

P: Co sie dzieje gdy cel wygasnie (nie zostanie trafiony)?

O: W onUpdate sprawdzam totalTime - spawnTime > TARGET_LIFETIME. Jezeli tak: -1 zycie, zeruje serie, chowam sfere (setVisible false), usuwam cel z listy. Gdy zycia spadna do 0 - koniec gry.

Gdzie: Gameplay.cpp -> onUpdate() (warunek totalTime - spawnTime > TARGET_LIFETIME)

P: Jak dziala serie trafien i bonus zycia?

O: Kazde trafienie zwieksza streak_, pudlo/wygasniecie zeruje. Co 5 trafien z rzędu (streak_ % 5 == 0) dodaje +1 zycie, max do 9. bestStreak_ pamietä rekord i pokazuje go na ekranie Game Over.

Gdzie: Gameplay.cpp -> onShoot() (++streak_, if streak_%5==0 ++lives_)

Constants / Header / CMake

P: Dlaczego stale sa constexpr w naglowku, a nie #define?

O: constexpr ma typ i zasięg (namespace sg), jest bezpieczniejsze niz #define (zwykła podmiana tekstu, bez kontroli typu). W C++17 constexpr w namespace jest niejawnie inline, więc nagłówek można włączać w wielu plikach .cpp bez błędu wielokrotnej definicji.

Gdzie: Constants.h -> namespace sg { constexpr ... }

P: Gdzie zmienilbym trudnosc gry?

O: W Constants.h - wszystkie liczby balansu sa tam: START_LIVES (zycia), TARGET_LIFETIME (czas na cel), PLAYER_MOVE_SPEED, MAX_MOVE_RANGE (jak szybko/daleko rusza sie cel), HIT_TOLERANCE. Zmiana jednej liczby zmienia trudnosc bez ruszania logiki.

Gdzie: Constants.h -> START_LIVES, TARGET_LIFETIME, itd.

P: Po co rozdzielacie .h (deklaracja) od .cpp (implementacja)?

O: Nagłówek to 'kontrakt' - mówi jakie są klasy i metody. Implementacja jest osobno. Dzięki temu 4 osoby mogą pisać różne pliki .cpp tej samej klasy ShootingGallery równolegle, nie wchodzić sobie w drogę, a kompilator wie z nagłówka, co istnieje.

Gdzie: ShootingGallery.h (deklaracja); Room/Obstacles/Targets/Gameplay.cpp (implementacja)

P: Jak to mozliwe, ze jedna klasa jest w 4 plikach .cpp?

O: C++ pozwala definiować metody klasy w różnych plikach .cpp - wystarczy poprzedzić je nazwą klasy (ShootingGallery::buildRoom itd.). Wszystkie .cpp włączają ten sam nagłówek. Linker skleja je w jeden obiekt. Dzięki temu podzielił pracę na 4 osoby.

Gdzie: ShootingGallery.h; 4 pliki .cpp z metodami ShootingGallery::...

P: Dlaczego assets kopiujecie osobnym targetem z ALL, a nie POST_BUILD?

O: POST_BUILD odpala się tylko gdy exe jest przebudowywane. Jak dodam nowy plik BMP, ale kod się nie zmienił, exe nie jest linkowane i kopiowanie się nie uruchamia. Osobny target z ALL wykonuje się przy KAŻDYM buildzie, więc nowe tekstury zawsze trafiają obok exe. To był realny problem, który naprawilem.

Gdzie: CMakeLists.txt -> add_custom_target(copy_assets ALL ...)

P: Jak CMake radzi sobie z roznymi systemami?

O: Sprawdzam zmienne APPLE / WIN32. Na Windows używam vcpkg lub MSYS2 (ucrt64 - dodał kolega), na macOS frameworka OpenGL i Homebrew, na Linux find_package(GLUT). Dzięki temu ten sam projekt kompiluje się na 3 systemach bez zmian w kodzie.

Gdzie: CMakeLists.txt -> if(APPLE)/elseif(WIN32)/else()

main.cpp

P: Co dokładnie robi main.cpp?

O: Tworzy Engine, wola init/setWindowParams/setGraphicsParams (okno + GL), tworzy obiekt gry ShootingGallery (PO setGraphicsParams - bo potrzebuje kontekstu GL na tekstury), rejestruje callbacki gry (update/shoot/hud/reset) i wola engine.run(). To cała integracja w ~20 liniach.

Gdzie: main.cpp -> int main()

P: Dlaczego gra jest tworzona po setGraphicsParams?

O: Bo konstruktor gry generuje tekstury proceduralne i wczytuje BMP, a to wymaga aktywnego kontekstu OpenGL. Kontekst powstaje dopiero w setGraphicsParams (glutCreateWindow). Gdyby gra powstała wcześniej, tekstury nie zostałyby utworzone.

Gdzie: main.cpp (kolejnosc: setGraphicsParams -> ShootingGallery game(engine))